

Hands-on 1: Introduction to HQL and JPQL

Overview

In Java enterprise applications, Object-Relational Mapping (ORM) frameworks such as Hibernate and Java Persistence API (JPA) simplify database interactions by mapping Java objects to database tables. To query these objects instead of directly writing SQL, Hibernate provides Hibernate Query Language (HQL) and JPA provides Java Persistence Query Language (JPQL).

Both HQL and JPQL are object-oriented query languages that allow developers to work with Java entity classes rather than database tables, improving readability, maintainability, and portability.

What is HQL?

Hibernate Query Language (HQL) is the query language provided by the Hibernate framework.

Features

- Object-oriented query language.
- Queries Java entity classes instead of database tables.
- Uses entity properties instead of column names.
- Similar syntax to SQL.
- Supports SELECT, UPDATE, DELETE, and INSERT operations.
- Supports Hibernate-specific features.
- Works with Hibernate Session API.

Example

```
String hql = "FROM Employee e WHERE e.salary > :minSalary";
```

```
Query<Employee> query = session.createQuery(hql, Employee.class);  
query.setParameter("minSalary", 50000);
```

```
List<Employee> employees = query.list();
```

What is JPQL?

Java Persistence Query Language (JPQL) is the standard query language defined by the JPA specification.

Features

- Standard query language for JPA.
- Database-independent.
- Works with entity objects and their fields.
- Uses EntityManager.
- Supports SELECT, UPDATE, and DELETE.
- Does not support INSERT.

Example

```
String jpql = "SELECT e FROM Employee e WHERE e.salary > :minSalary";
```

```
Query query = entityManager.createQuery(jpql);
```

```
query.setParameter("minSalary", 50000);
```

```
List<Employee> employees = query.getResultList();
```

Similarities Between HQL and JPQL

- Both are object-oriented query languages.
- Both query Java entity classes instead of database tables.
- Syntax is similar to SQL.
- Both support:
 - SELECT
 - UPDATE
 - DELETE
- Both support:
 - Joins
 - Group By
 - Order By

- Aggregate Functions
- Subqueries
- Named Queries
- Parameter Binding
- Both return Java objects instead of database rows.

Example: UPDATE Query

```
String jpql = "UPDATE Employee e SET e.salary = e.salary + 5000  
WHERE e.department.name = 'IT'";
```

```
Query query = entityManager.createQuery(jpql);  
query.executeUpdate();
```

Example: DELETE Query

```
String jpql = "DELETE FROM Employee e WHERE e.id = :id";
```

```
Query query = entityManager.createQuery(jpql);  
query.setParameter("id", 101);  
query.executeUpdate();
```

Difference Between HQL and JPQL

HQL	JPQL
Hibernate-specific	JPA Standard
Supports INSERT	Does not support INSERT
Uses Hibernate Session	Uses EntityManager
Includes Hibernate-specific features	Portable across JPA providers
All JPQL queries are valid HQL	JPQL is a subset of HQL

When to Use HQL

- Working specifically with Hibernate.
- You need Hibernate-specific features.
- You want to use INSERT statements.

When to Use JPQL

- Building portable applications.
- Using standard JPA.
- Supporting different JPA providers like Hibernate, EclipseLink, or OpenJPA.